UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Chapter 6: Class Relationships

SECJ2154: Object-Oriented Programming
Faculty of Computing (FC), UTM

Note: This slide is based on Dr. Norazah Yusof's slide
Updated by Shahida Sulaiman, 2025
Other sources are cited accordingly
Reminder: Do not upload this slide to external platforms

Innovating Solutions

UTM JOHOR BAHRU

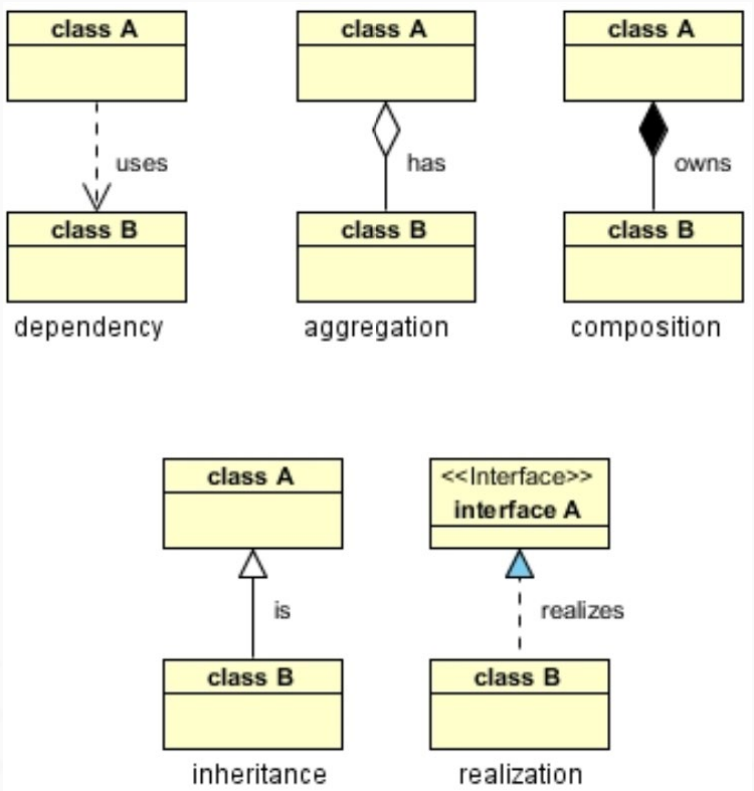
Contents

- **Introduction**
  - Types of Relationship
- **Association**
- **Aggregation**
- **Composition**
- **Inheritance** (Details in Chapter 7)

Types of Relationships among Classes

INTRODUCTION

- **Association:** A general relationship where one class uses or communicates with another
- **Aggregation:** A "whole-part" relationship where the part can exist independently of the whole.
- **Composition:** A strong form of aggregation where the part cannot exist without the whole.
- **Inheritance:** A "is-a" relationship where one class is a specialized form of another. (To be elaborated in Chap. 7)

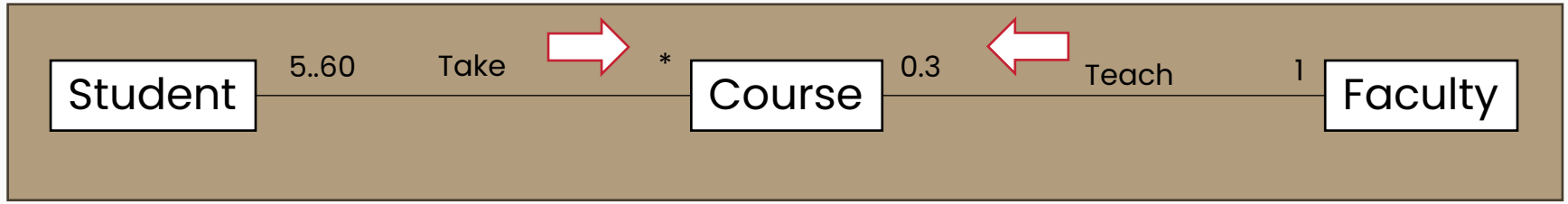


https://singhblogging.wordpress.com/2018/05/14/relationships-aggregation-composition-and-uml-class-diagram/

ASSOCIATION

Association

- Association represents a **general binary relationship** that describes an activity between two classes.
- An association is usually represented as a **data field** in the class.



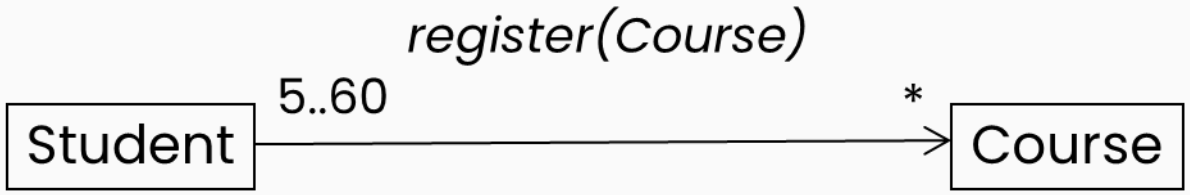

```
public class Student {
  /** Data fields */
  private Course[]
    courseList;

  /** Constructors */
  /** Methods */
}
```

```
public class Course {
  /** Data fields */
  private Student[]
    classList;
  private Faculty faculty

  /** Constructors */
  /** Methods */
}
```

```
public class Faculty {
  /** Data fields */
  private Course[]
    courseList;

  /** Constructors */
  /** Methods */
}
```

Association

- Example: **Student** register **Course**

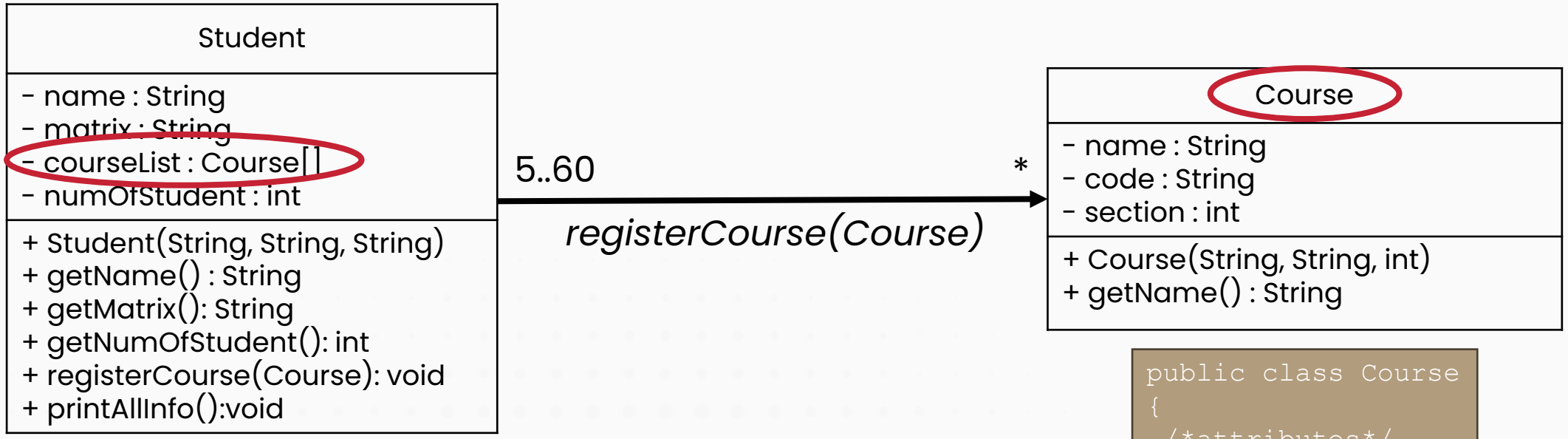


- The arrow is optional to specify navigability.
- No arrow implies bidirectional navigability

ASSOCIATION

Association

- An association is usually represented as a data field in the class.



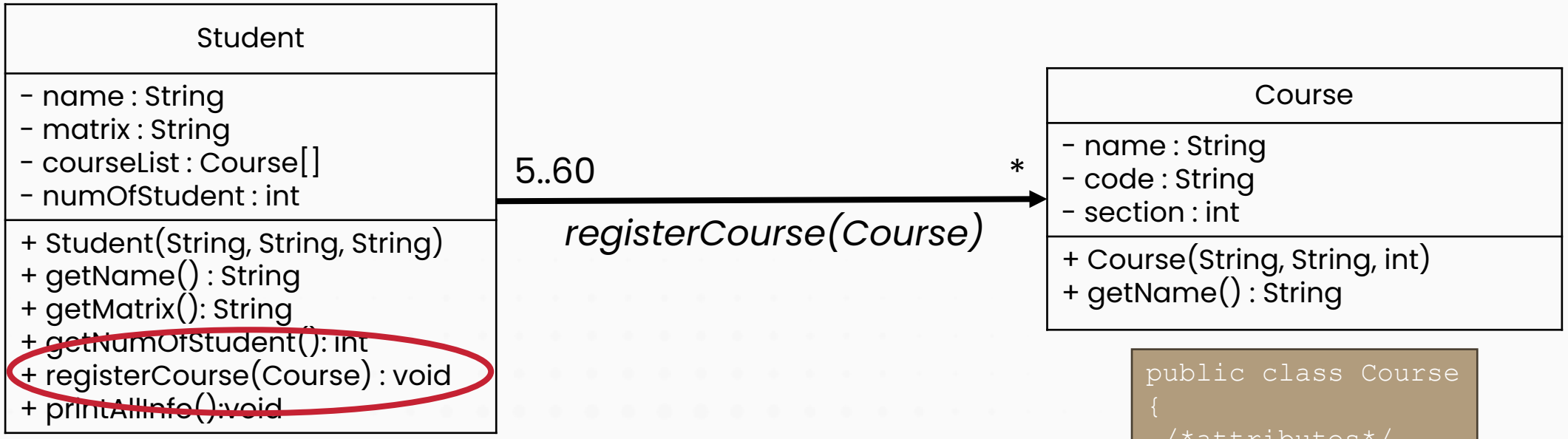

```
public class Student
{ /*attributes*/
  private Course[] courseList;
/*methods*/}
```

```
public class Course
{
 /*attributes*/
 /*methods*/
}
```

ASSOCIATION

Association

- The relationship allows objects to call methods in other objects.



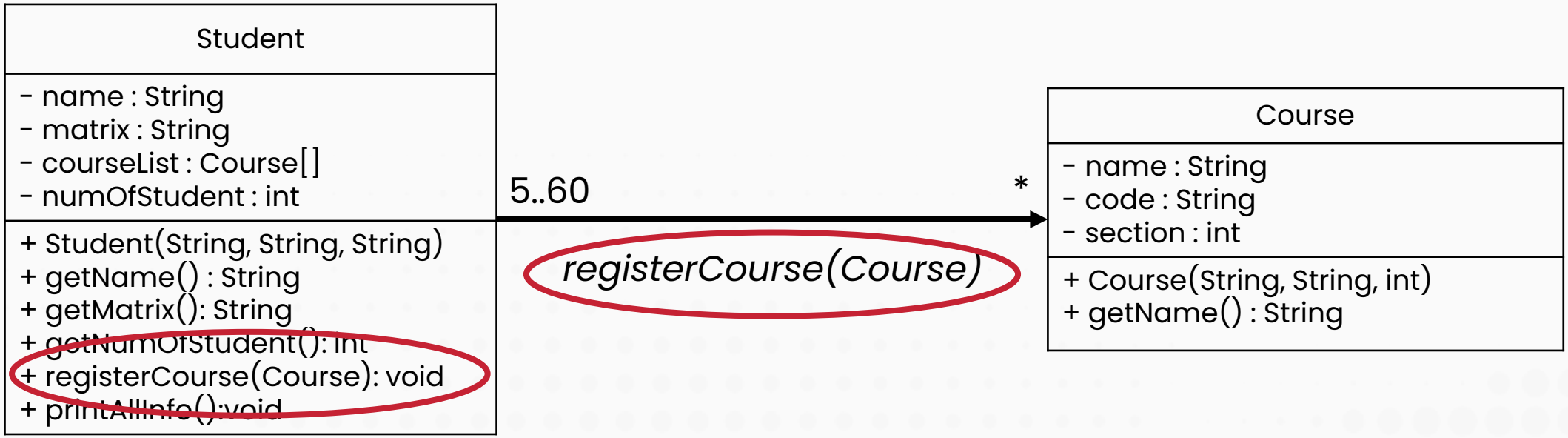

```
public class Student
{ /*attributes*/
  private Course[] courseList;
/*methods*/}
```

```
public class Course
{
 /*attributes*/
 /*methods*/
}
```

ASSOCIATION

Association

- This is the same as an object sending another object a **message**.
- Therefore, it is implemented as **object references**.



ASSOCIATION

Association Example 1: Student register Course

- Create 3 java files:
  - TestAssociate.java
  - Student.java
  - Course.java
- Create 3 `Course` objects
- Create 2 `Student` objects
- Student registers 2 courses
- Print all students info

```
public class TestAssociate {
    public static void main(String[] args) {
        // Create some courses
        Course c1 = new Course("CS101", "Intro to
Programming");
        Course c2 = new Course("CS102", "Data
Structures");
        Course c3 = new Course("CS103", "Algorithms");

        // Create two students
        Student student1 = new Student("Alice", "S101");
        Student student2 = new Student("James", "S102");

        // Register students in courses
        student1.registerCourse(c1);
        student1.registerCourse(c2);

        student2.registerCourse(c1);
        student2.registerCourse(c3);

        // Print student info
        student1.printAllInfo();
        student2.printAllInfo();
    }
}
```

Association Example 1: Student register Course (Student class)

ASSOCIATION

```java
public class Student {
    private String name;
    private String matrix;
    private Course[] courseList; //association
    private int numOfCourses;

    public Student(String name, String matrix) {
        this.name = name;
        this.matrix = matrix;
        this.courseList = new Course[10]; // max 10
        this.numOfCourses = 0;
    }

    public String getName() {
        return name;
    }

    public String getMatrix() {
        return matrix;
    }
```

```java
    public void registerCourse(Course course) {
    //association
        if (numOfCourses < courseList.length) {
            courseList[numOfCourses] = course;
            numOfCourses++;
        } else {
            System.out.println("Cannot register:
course list is full for " + name);
        }
    }

    public void printAllInfo() {
        System.out.println(name + " (" + matrix + ")
is registered in:");
        for (int i = 0; i < numOfCourses; i++) {
            System.out.println("- " +
courseList[i]);
        }
        System.out.println();
    }
}
```

Association Example 1: Student register Course (Course class)

ASSOCIATION

```
public class Course {
    private String code;
    private String title;

    public Course(String code, String title) {
        this.code = code;
        this.title = title;
    }

    public String getCode() {
        return code;
    }

    public String getTitle() {
        return title;
    }

    @Override
    public String toString() {
        return code + " - " + title;
    }
}
```

Association Example 2: Lecturer teach Course

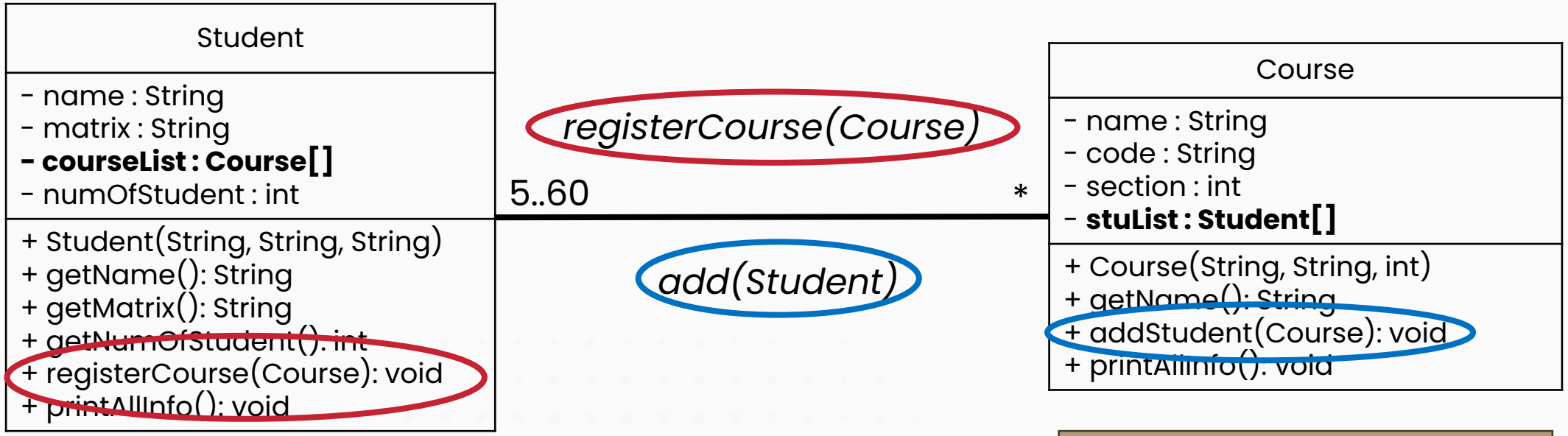
- Based on E.g. 1, write the Java code for the two classes
- Test the classes to show association relationships among them

```
public class Course {
 /*attributes*/
 /*methods*/
}
```

```
public class Lecturer {
  /*attributes*/
  private Course[] courseList;
  /*methods*/
}
```

Association Example 3: Bidirectional Relationship
Student register Course and Course add Student

ASSOCIATION




```
public class Student {
  /*attributes*/
  private Course[] courseList;
  /*methods*/
}
```

```
public class Course {
  /*attributes*/
  private Student[] stuList;
  /*methods*/
}
```

Association Example 3: Bidirectional Relationship
Student register Course and Course add Student

ASSOCIATION

```
public class TestAssociate2 {
    public static void main(String[] args) {
        // Courses
        Course1 c1 = new Course1("Intro to
Programming", "CS101", 1);
        Course1 c2 = new Course1("Data Structures",
"CS102", 1);
        Course1 c3 = new Course1("Algorithms",
"CS103", 1);
        Course1 c4 = new Course1("Operating
Systems", "CS104", 1);

        // Students
        Student1 s1 = new Student1("Ali", "S001");
        Student1 s2 = new Student1("Tan", "S002");
        Student1 s3 = new Student1("Sam", "S003");
```

```
 // Registration (bidirectional via addStudent)
        c1.addStudent(s1);
        c1.addStudent(s2);
        c2.addStudent(s1);
        c2.addStudent(s3);

        c3.addStudent(s2);

        c4.addStudent(s2);
        c4.addStudent(s3);

        // Only printing student info
        s1.printAllInfo();
        s2.printAllInfo();
        s3.printAllInfo();
    }
}
```

Association Example 3: Bidirectional Relationship
Student register Course (`Student1` Class)

ASSOCIATION

```java
public class Student1 {
    private String name;
    private String matrix;
    private Course1[] courseList; //association
    private int numOfCourse;

    public Student1(String n, String m) {
        name = n;
        matrix = m;
        courseList = new Course1[10]; //association
        numOfCourse = 0;
    }

    public String getName() {
        return name;
    }
```

```java
    public void register(Course1 c) { //association
        courseList[numOfCourse] = c;
        numOfCourse++;
    }

    public void printAllInfo() {
        System.out.println(name + " (" + matrix +
") is registered in:");
        for (int i = 0; i < numOfCourse; i++) {
            System.out.println("- " +
courseList[i]);
        }
        System.out.println();
    }
}
```

Association Example 3: Bidirectional Relationship
Course add Student (`Course1` Class)

ASSOCIATION

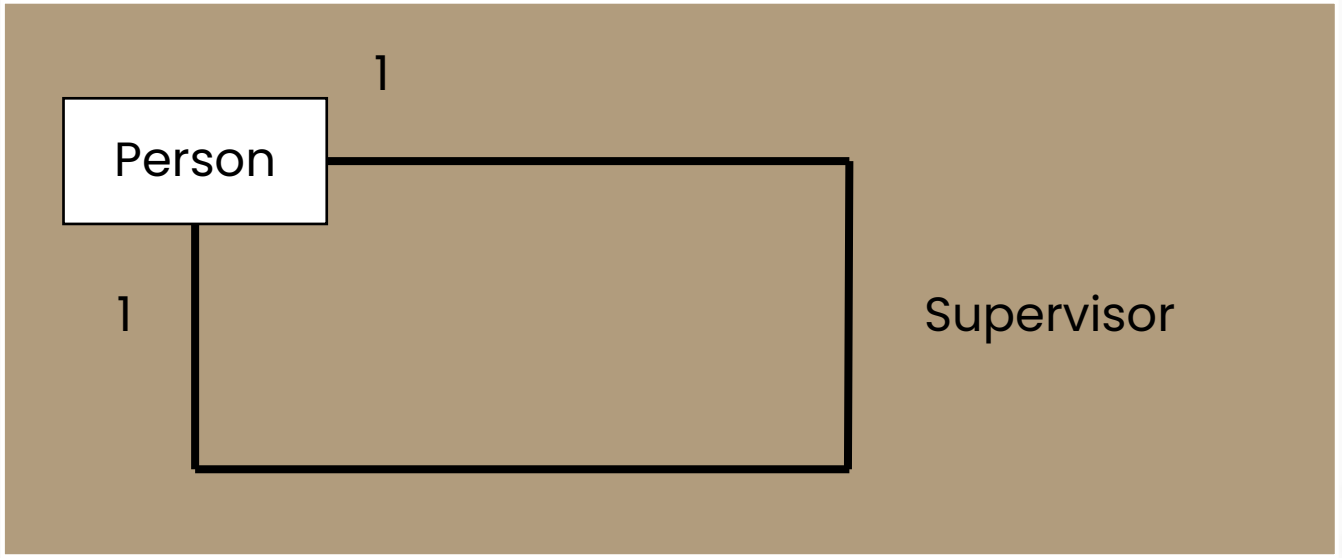
```
public class Course1 {
    private String name;
    private String code;
    private int section;
    private Student1[] stuList; //association
    private int numOfStudent;

    public Course1(String n, String c, int s) {
        name = n;
        code = c;
        section = s;
        stuList = new Student1[60]; // max 60
students
        numOfStudent = 0;
    }
```

```
    public String getName() {
        return name;
    }

    public void addStudent(Student1 st) {
    // bidirectional association
        stuList[numOfStudent] = st;
        numOfStudent++;
        st.register(this);
    }

    @Override
    public String toString() {
        return name + " (" + code + "-" + section +
")";
    }
}
```

Association Between Objects of the Same Class (Recursive Association)

ASSOCIATION

- Association may exist between **objects of the same class**.
- It is also known as recursive association or self-referential association
- For example, a **person** may **have** a **supervisor**, who is also a person



Recursive Association: Person Class

ASSOCIATION

```
public class TestPerson {
    public static void main(String[] args) {
        Person john = new Person("John");
        Person alice = new Person("Alice");
        Person bob = new Person("Bob");

        // Set supervisors
        john.setSupervisor(alice); // John's supervisor is Alice
        alice.setSupervisor(bob);  // Alice's supervisor is Bob

        // Print out the hierarchy
        System.out.println(john);
        System.out.println(alice);
        System.out.println(bob);
    }
}
```

Recursive Association: Person Class

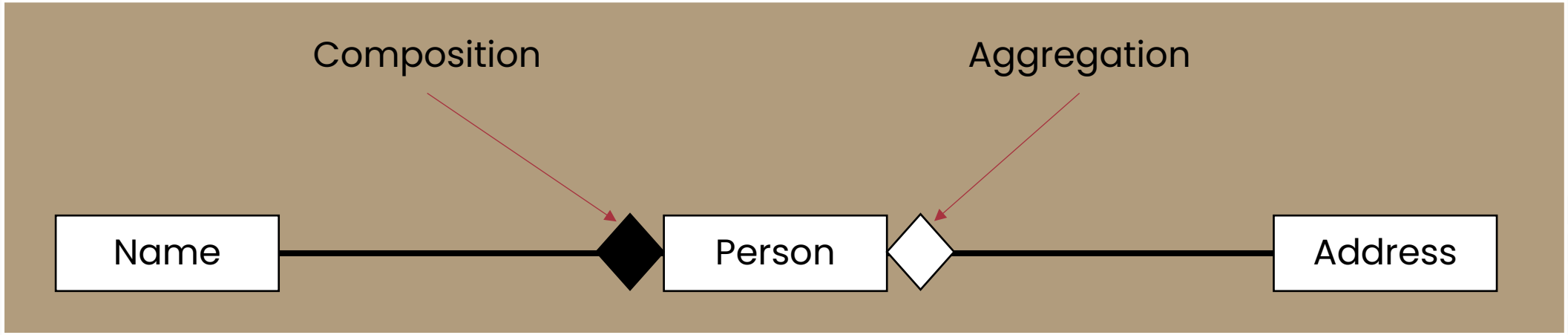
ASSOCIATION

```
public class Person {
    private String name;
    private Person supervisor; //Self-reference: supervisor is also a Person

    public Person(String name) {
        this.name = name;
        this.supervisor = null; // No supervisor by default
    }

    public void setSupervisor(Person supervisor) {
        this.supervisor = supervisor;
    }

    public Person getSupervisor() {
        return supervisor;
    }

        public String getName() {
        return name;
    }

    @Override
    public String toString() {
        if (supervisor != null) {
            return name + " (Supervisor: " + supervisor.getName() + ")";
        } else {
            return name + " (No Supervisor)";
        }
    }
}
```

AGGREGATION

Aggregation and Composition

- **Aggregation** is a special form of association, which represents an **ownership relationship** between two classes.
- Aggregation models the **has-a relationship**.
- If an object is **exclusively owned** by an aggregated object, the relationship between the object and its aggregated object is referred to as **composition**.



Representing Aggregation in Classes

- An **aggregation** relationship is usually represented as a **data field** in the aggregated class.
- Example a person has name and address represented as three different classes as shown below.

```
public class Name {
   /** Data fields */
   /** Constructors */
   /** Methods */
}
```

```
public class Person {
   /** Data fields */
   private Name name;
   private Address address;

   /** Constructors */
   /** Methods */
}
```

```
public class Address {
   /** Data fields */
   /** Constructors */
   /** Methods */
}
```

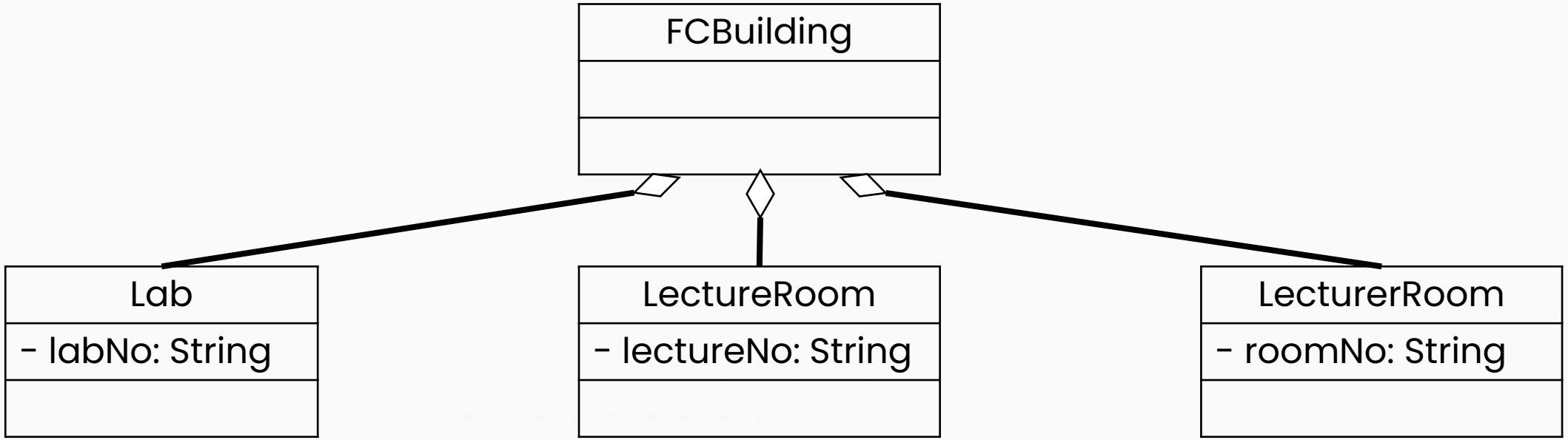
Inner Classes Translation

- If **`Name` or `Address`** is used in the **`Person` class only**, they can be declared as an **inner class** in `Person`.
- For example:

```
public class Person {
  private Name name;
  private Address address;
  ...

  class Name {
  ...
  }

  class Address {
  ...
  }
}
```

AGGREGATION

Aggregation Example: FC building has Lab, LectureRoom and LecturerRoom




```
public class FCBuilding {
    private Lab lab1;
    private LectureRoom lect;
    private LecturerRoom lectr;
}
```

Aggregation relationship is represented as a data field in the aggregated class.

Aggregation Example: FC building has Lab, LectureRoom and LecturerRoom

ASSOCIATION

- Create 4 java files:
  - TestFCBuilding.java
  - Lab.java
  - LectureRoom.java
  - LecturerRoom.java
- `FCBuilding` class aggregates multiple room types:
  - Lab
  - LectureRoom
  - LecturerRoom

```
public class TestFCBuilding {
    public static void main(String[] args) {
    //Create individual room instances (can exist
independently)
        Lab lab = new Lab("MPK1");
        LectureRoom lecture = new LectureRoom("BK1");
        LecturerRoom lecturerRoom = new
LecturerRoom("305");

        // Aggregate them into FCBuilding
        FCBuilding building = new FCBuilding(lab,
lecture, lecturerRoom);

        System.out.println(building);
    }
}
```

Aggregation Example: FC building has Lab, LectureRoom and LecturerRoom (FCBuilding & Lab Class)

ASSOCIATION

```java
public class FCBuilding {
    private Lab lab1;              // Aggregated Lab
    private LectureRoom lect;      // Aggregated Lecture Room
    private LecturerRoom lectr;    // Aggregated Lecturer Room

    public FCBuilding(Lab lab1, LectureRoom lect, LecturerRoom
lectr) {
        this.lab1 = lab1;
        this.lect = lect;
        this.lectr = lectr;
    }

    @Override
    public String toString() {
        return "FC Building contains:\n" +
               lab1 + "\n" +
               lect + "\n" +
               lectr;
    }
}
```

```java
public class Lab {
    private String labNo;

    public Lab(String labNo) {
        this.labNo = labNo;
    }

    public String getLabNo() {
        return labNo;
    }

    @Override
    public String toString() {
        return "Lab No: " + labNo;
    }
}
```

Aggregation Example: FC building has Lab, LectureRoom and LecturerRoom (LectureRoom & LecturerRoom Class)

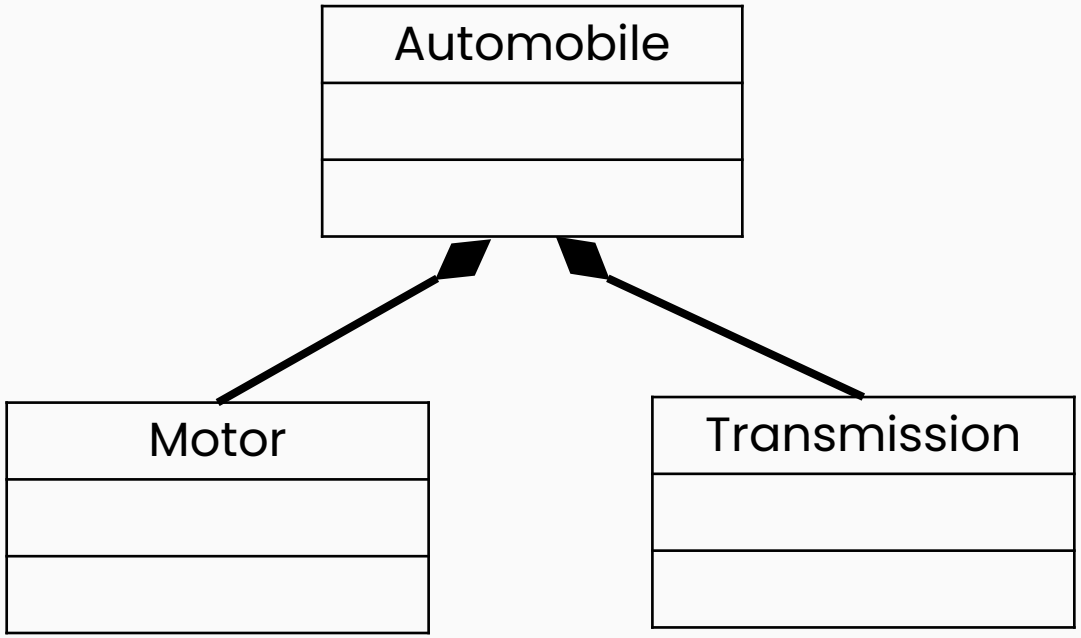
ASSOCIATION

```
public class LectureRoom {
    private String lectureNo;

    public LectureRoom(String lectureNo) {
        this.lectureNo = lectureNo;
    }

    public String getLectureNo() {
        return lectureNo;
    }

    @Override
    public String toString() {
        return "Lecture Room No: " +
lectureNo;
    }
}
```

```
public class LecturerRoom {
    private String roomNo;

    public LecturerRoom(String roomNo) {
        this.roomNo = roomNo;
    }

    public String getRoomNo() {
        return roomNo;
    }

    @Override
    public String toString() {
        return "Lecturer Room No: " + roomNo;
    }
}
```

COMPOSITION

Composition

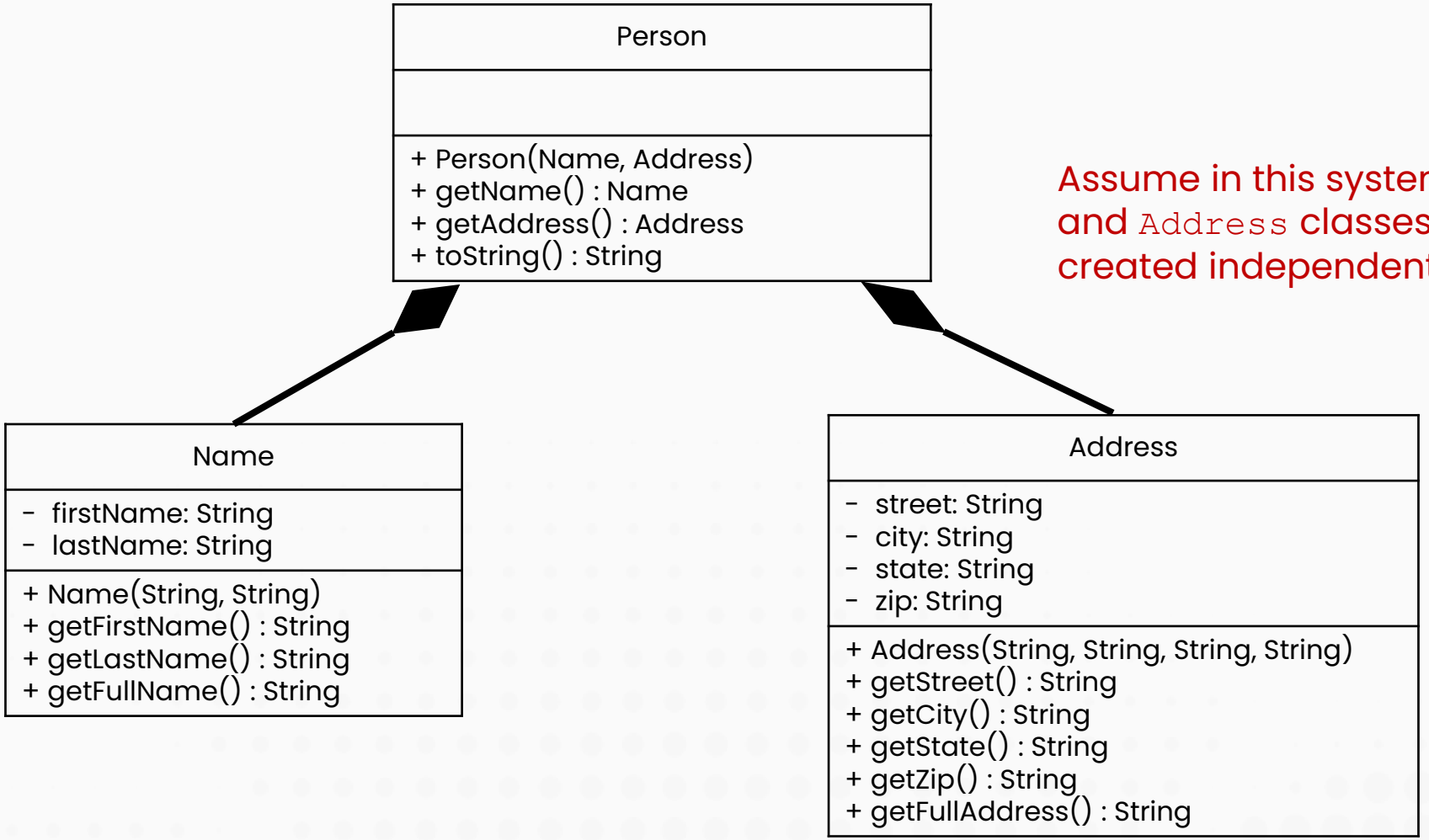


- Form of aggregation
- For a relationship "**contains a**"
- Strong ownership / binding
- Parts belong to one whole
- Parts do not change during execution

```
public class Automobile
{
    private final Transmission drive = new Transmission();
    private final Motor = new Motor();
}
```

COMPOSITION

Composition Example:
A Person has a Name and an Address



Assume in this system, both `Name` and `Address` classes cannot be created independently

Composition Example 1:
Person has a name and an address

ASSOCIATION

- Create 4 java files:
  - TestPerson.java
  - Person.java
  - Name.java
  - Address.java
- Create 1 `Name` object
- Create 1 `Address` object
- Create 1 `Person` object

```
// Test class to demonstrate composition in Person class
public class TestPerson {
    public static void main(String[] args) {
        // Compose Name, Address inside Person (composition)
        Person per1 = new Person("Ali", "Ahmad", "Jln. 12",
"Johor Bahru", "Johor", "81310");

        // Display the person's information
        System.out.println(per1);
    }
}
```

Composition Example 1:
Person has a name and an address (Person Class)

ASSOCIATION

```
// Person class that uses composition to include Name and Address
public class Person {
    private Name name;    // Composition: Person creates and owns Name
    private Address address; // Composition: Person creates and owns Address

    // Constructor to initialize name and address using their parts
    public Person(String firstName, String lastName, String street, String city, String state, String zip) {
        this.name = new Name(firstName, lastName);            // created inside => composition
        this.address = new Address(street, city, state, zip); // created inside => composition
    }

    public Name getName() {
        return name;
    }

    public Address getAddress() {
        return address;
    }

    @Override
    public String toString() {
        return "Name: " + name.getFullName() + "\n" +
               "Address: " + address.getFullAddress();
    }
}
```

Composition Example 1:
Person has a name and an address (Name Class)

ASSOCIATION

```java
// Name class represents a person's name
public class Name {
    private String firstName;
    private String lastName;

    // Constructor to initialize first and last name
    public Name(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    // Getter for first name
    public String getFirstName() {
        return firstName;
    }
```

```java
    // Getter for last name
    public String getLastName() {
        return lastName;
    }

    // Returns the full name
    public String getFullName() {
        return firstName + " " + lastName;
    }

    // String representation of Name object
    @Override
    public String toString() {
        return getFullName();
    }
}
```

Composition Example 1:
Person has a name and an address (Address Class)

ASSOCIATION

```
// Address class represents a person's address
public class Address {
    private String street;
    private String city;
    private String state;
    private String zip;

    // Constructor to initialize all address fields
    public Address(String street, String city, String
state, String zip) {
        this.street = street;
        this.city = city;
        this.state = state;
        this.zip = zip;
    }

    // Getter methods for each attribute
    public String getStreet() {
        return street;
    }

    public String getCity() {
        return city;
    }
```

```
 public String getState() {
        return state;
    }

    public String getZip() {
        return zip;
    }

    // Returns full formatted address
    public String getFullAddress() {
        return street + ", " + city + ", " +
state + " " + zip;
    }

    // String representation of Address object
    @Override
    public String toString() {
        return getFullAddress();
    }
}
```

Aggregation vs. Composition

SUMMARY

- Consider two simplified examples from related systems:
  - A team has players, and the players can exist independently, as they can play with other teams
  - A book has chapters, and the chapters cannot exist without a book

Feature	Aggregation (Team-Player)	Composition (Book-Chapter)
Object creation	Pass in from outside	Created internally
Ownership	No (shared with other objects)	Yes (exclusive to its own object)
Lifecycle	Independent	Bound to the whole (Book)

Aggregation Example: Team has Player

SUMMARY

```java
public class TestAggregation {
    public static void main(String[] args) {
        Player p = new Player("Ali");
        Team t1 = new Team(p);
        //Same player could be on multiple teams
        Team t2 = new Team(p);

        t1.showPlayer();
        t2.showPlayer();
    }
}
```

```java
// Class that can exist independently
class Player {
    private String name;

    public Player(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

// Aggregation: Team uses Player but does not own it
class Team {
    private Player player;

    public Team(Player player) {
        this.player = player;
        // Player is passed in, not created here
    }

    public void showPlayer() {
        System.out.println("Team has player: " +
player.getName());
    }
}
```

Composition Example: Book has Chapter

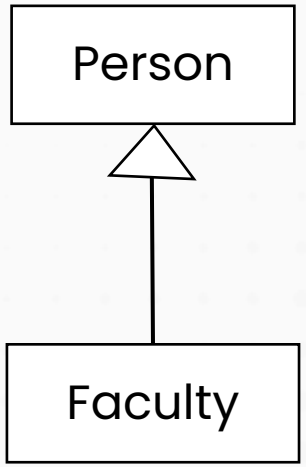
SUMMARY

```
public class TestComposition {
    public static void main(String[] args) {
        Book b1 = new Book("Introduction to
Java");
        b1.showChapter();
    }
}
```

```
// Chapter class is not meaningful without Book
class Chapter {
    private String title;

    public Chapter(String title) {
        this.title = title;
    }

    public String getTitle() {
        return title;
    }
}

// Composition: Book owns Chapters
class Book {
    private Chapter chapter;

    public Book(String chapterTitle) {
        this.chapter = new Chapter(chapterTitle);
        // Book creates the chapter
    }

    public void showChapter() {
        System.out.println("Book contains chapter: "
+ chapter.getTitle());
    }
}
```

Inheritance (Details will be in Chapter 7)

- Inheritance models the **is-an-extension-of** relationship between two classes.
- Example: `Faculty` class extends `Person` class




```
public class Faculty extends Person {
  /** Data fields */
  /** Constructors */
  /** Methods */
}
```

Exercise

SELF-TEST

- What is the appropriate **relationship** for the following classes:
  - Company and Employee (assume in a Freelancer app)
  - Faculty and Course (consider service courses shared by a few faculties)
  - Customer and Product
  - Student and Matric Number
  - Employee and Address
  - House and Door